

# Implementation Plan: Safe Feature Deployment & Rollback

Project: AttendTrack • Environment: Next.js + Firebase Hosting (Spark Plan)

## 1. Architecture & Workflow Overview

This plan details the standard engineering workflow to build, test, and release new features (such as external API integrations) with zero risk of downtime or breaking live production users. By utilizing Git feature branching and Firebase Hosting Preview Channels, you can test isolated previews live on real devices before merging to production.

|                            |                                                                           |
|----------------------------|---------------------------------------------------------------------------|
| Step 1: Local Branching    | Create isolated branch feature/new-api-integration                        |
| Step 2: Preview Channel    | Deploy to hosting:channel:deploy preview-new-feature for live testing     |
| Step 3: Production Release | Merge branch to main and deploy live via firebase deploy                  |
| Step 4: Emergency Rollback | One-click instant rollback (< 5 seconds) via Firebase Console if required |

## 2. Step-by-Step Implementation Guide

### Phase 1: Create an Isolated Git Branch

Never write experimental API code directly on the main branch. Always work in an isolated feature branch:

```
git checkout main
git pull origin main
git checkout -b feature/new-api-integration
```

### Phase 2: Deploy to a Firebase Preview Channel (Zero Risk)

Firebase Hosting Preview Channels allow you to deploy a live testing URL without touching production:

```
npm run build
npx firebase-tools hosting:channel:deploy preview-api-feature
```

**Result:** Firebase generates a unique URL (e.g., <https://testchecker-4caeb--preview-api-feature-xyz.web.app>). You can share and test this URL on mobile and web with full fidelity.

### Phase 3: Promote to Production

Once all test cases pass on the preview channel, merge to main and push to live production:

```
git add .
git commit -m "Add and verify new API integration"
git checkout main
git merge feature/new-api-integration
git push origin main
npm run build
npx firebase-tools deploy --only hosting
```

## Phase 4: Emergency Rollback Strategy

If a critical bug or API failure occurs in production, use one of three recovery methods:

### Method 1: Instant 1-Click Rollback (Recommended — < 5 Seconds)

1. Go to Firebase Console > Hosting > Release History.
2. Locate the previous working version.
3. Click  and select **Rollback**. Site reverts instantly with zero downtime.

### Method 2: Firebase CLI Command Line Rollback

```
npx firebase-tools hosting:releases:list
npx firebase-tools hosting:clone testchecker-4caeb:PREVIOUS_VERSION_ID testchecker-4caeb:live
```

### Method 3: Git Revert Rollback

```
git revert HEAD
git push origin main
npm run build
npx firebase-tools deploy --only hosting
```

## 3. Summary Command Reference

| Action                 | Command Line                                                                                       |
|------------------------|----------------------------------------------------------------------------------------------------|
| Create Feature Branch  | <code>git checkout -b feature/my-new-feature</code>                                                |
| Deploy Preview Channel | <code>npm run build &amp;&amp; npx firebase-tools hosting:channel:deploy preview-name</code>       |
| Deploy to Production   | <code>npm run build &amp;&amp; npx firebase-tools deploy --only hosting</code>                     |
| Delete Preview Channel | <code>npx firebase-tools hosting:channel:delete preview-name</code>                                |
| CLI Emergency Rollback | <code>npx firebase-tools hosting:clone testchecker-4caeb:PREVIOUS_ID testchecker-4caeb:live</code> |